Triggers and Constraints

Travel Itinerary and Expense Organizer

Complete reference for every enforced rule in the schema

Database: `travel_itinerary_db`

Date: 26 October 2026

Source of truth: generated from `information_schema` on a live MySQL 8.4 server

> **What is in this document.** Every rule the database enforces, taken straight from the running server rather than retyped: **31 triggers**, **27 CHECK constraints**, **22 ENUM columns**, **19 UNIQUE constraints** and **42 foreign keys**. Each entry says what it protects and shows the SQL that implements it.

1. Why there are several layers of rules

A single mechanism cannot express every rule this schema needs, so enforcement is spread over four layers. Each one covers what the layer below it cannot. Understanding the split is the quickest way to read the rest of this document.

Layer	What it can express	Why the others cannot do it
Foreign keys 42 of them	That a referenced row exists, and what happens to children when a parent is removed.	They check existence, not values. They cannot say "the guests must fit the room" or "the dates must be free".
CHECK constraints 27 of them	That a single row's values are sensible: ranges, positive amounts, an end after a start.	They see one row at a time. They cannot look at another table or at other rows.
ENUM and UNIQUE 22 and 19	That a column holds one of a fixed list, or that a combination of values appears once.	ENUM covers a fixed vocabulary but not numeric ranges. UNIQUE covers duplicates but not meaning.
Triggers 31 of them	Rules that span tables or rows: capacity against a booking, overlaps, permissions, audit trails.	Nothing below can read other tables, so this is the only place such rules can live.

Reading the trigger names. The suffix says when the trigger runs: `_bi` is before insert, `_bu` is before update, `_ai` is after insert, `_au` is after update and `_ad` is after delete. Most validation triggers exist as a matched pair, one for insert and one for update, so a row cannot be made invalid by editing it later.

2. Triggers (31)

Grouped by the script that creates them. A rejected write raises MySQL error 1644 (`SIGNAL SQLSTATE '45000'`) carrying the message shown in the SQL block.

A. Data-integrity triggers — 01_travel_itinerary_v2_full.sql, 12 triggers

trg_days_validate_bi BEFORE INSERT on itinerary_days

Keeps a day in step with its trip. `day_date` must equal `start_date + day_number - 1`, and no day may fall outside the trip.

```
BEGIN DECLARE v_start DATE; DECLARE v_end DATE; SELECT start_date, end_date INTO v_start, v_end FROM trips
WHERE trip_id = NEW.trip_id; IF v_start IS NOT NULL THEN IF NEW.day_date <> DATE_ADD(v_start, INTERVAL
NEW.day_number - 1 DAY) THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'day_date must equal trip
start_date + day_number - 1'; END IF; IF NEW.day_date > v_end THEN SIGNAL SQLSTATE '45000' SET
MESSAGE_TEXT = 'day_date is outside the trip dates'; END IF; END IF; END
```

trg_days_validate_bu BEFORE UPDATE on itinerary_days

Same rule on update, so a day cannot be moved out of line after it is created.

```
BEGIN DECLARE v_start DATE; DECLARE v_end DATE; SELECT start_date, end_date INTO v_start, v_end FROM trips
WHERE trip_id = NEW.trip_id; IF v_start IS NOT NULL THEN IF NEW.day_date <> DATE_ADD(v_start, INTERVAL
NEW.day_number - 1 DAY) THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'day_date must equal trip
start_date + day_number - 1'; END IF; IF NEW.day_date > v_end THEN SIGNAL SQLSTATE '45000' SET
MESSAGE_TEXT = 'day_date is outside the trip dates'; END IF; END IF; END
```

trg_items_no_overlap_bi BEFORE INSERT on itinerary_items

Two timed activities on the same day may not overlap. Untimed items are ignored, so a note can sit alongside anything.

```
BEGIN DECLARE v_cnt INT; IF NEW.start_time IS NOT NULL AND NEW.end_time IS NOT NULL THEN SELECT COUNT(*)
INTO v_cnt FROM itinerary_items i WHERE i.day_id = NEW.day_id AND i.item_id <> COALESCE(NEW.item_id, 0)
AND i.start_time IS NOT NULL AND i.end_time IS NOT NULL AND NEW.start_time < i.end_time AND NEW.end_time >
i.start_time; IF v_cnt > 0 THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Timed items on the same day
must not overlap'; END IF; END IF; END
```

trg_items_no_overlap_bu BEFORE UPDATE on itinerary_items

Same rule on update. The check excludes the row being changed, so an item never clashes with itself.

```
BEGIN DECLARE v_cnt INT; IF NEW.start_time IS NOT NULL AND NEW.end_time IS NOT NULL THEN SELECT COUNT(*)
INTO v_cnt FROM itinerary_items i WHERE i.day_id = NEW.day_id AND i.item_id <> COALESCE(NEW.item_id, 0)
AND i.start_time IS NOT NULL AND i.end_time IS NOT NULL AND NEW.start_time < i.end_time AND NEW.end_time >
i.start_time; IF v_cnt > 0 THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Timed items on the same day
must not overlap'; END IF; END IF; END
```

trg_exp_item_trip_bi BEFORE INSERT on expenses

An expense may only point at an itinerary item from the same trip. A foreign key alone would allow attaching an expense to another trip's activity.

```
BEGIN DECLARE v_item_trip INT; IF NEW.item_id IS NOT NULL THEN SELECT d.trip_id INTO v_item_trip FROM
itinerary_items i JOIN itinerary_days d ON d.day_id = i.day_id WHERE i.item_id = NEW.item_id; IF
v_item_trip <> NEW.trip_id THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Expense item must belong to
the same trip as the expense'; END IF; END IF; END
```

trg_exp_item_trip_bu BEFORE UPDATE on expenses

Same rule on update, so the link cannot be moved across trips later.

```
BEGIN DECLARE v_item_trip INT; IF NEW.item_id IS NOT NULL THEN SELECT d.trip_id INTO v_item_trip FROM
itinerary_items i JOIN itinerary_days d ON d.day_id = i.day_id WHERE i.item_id = NEW.item_id; IF
v_item_trip <> NEW.trip_id THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Expense item must belong to
the same trip as the expense'; END IF; END IF; END
```

trg_res_validate_bi BEFORE INSERT on hotel_reservations

Five rules for a hotel booking: the room must exist and not be under maintenance; the guests must fit the room type capacity; the stay must fall inside the trip dates; the booker must be the trip owner or an editor; and the room must be free for those dates.

```
BEGIN DECLARE v_room_status VARCHAR(20); DECLARE v_cap INT; DECLARE v_ts DATE; DECLARE v_te DATE; DECLARE
v_cnt INT; IF NEW.status <> 'cancelled' THEN -- room must exist, not be under maintenance, and fit the
guests SELECT r.status, rt.max_occupancy INTO v_room_status, v_cap FROM rooms r JOIN room_types rt ON
rt.room_type_id = r.room_type_id AND rt.hotel_id = r.hotel_id WHERE r.room_id = NEW.room_id; IF
v_room_status = 'maintenance' THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Room is under maintenance
and cannot be reserved'; END IF; IF NEW.num_guests > v_cap THEN
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Number of guests exceeds the room type capacity'; END IF; --
stay must lie inside the trip dates (check-out may be the day after the last day) SELECT start_date,
end_date INTO v_ts, v_te FROM trips WHERE trip_id = NEW.trip_id; IF NEW.check_in < v_ts OR NEW.check_out >
DATE_ADD(v_te, INTERVAL 1 DAY) THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Stay must be inside the
trip dates'; END IF; -- only an owner or editor of the trip may book SELECT COUNT(*) INTO v_cnt FROM
trip_members WHERE trip_id = NEW.trip_id AND user_id = NEW.booked_by AND role IN ('owner','editor'); IF
v_cnt = 0 THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Only a trip owner or editor can book a room';
END IF; -- no double booking of the same room on overlapping nights SELECT COUNT(*) INTO v_cnt FROM
hotel_reservations h WHERE h.room_id = NEW.room_id AND h.status <> 'cancelled' AND h.reservation_id <>
COALESCE(NEW.reservation_id, 0) AND NEW.check_in < h.check_out AND NEW.check_out > h.check_in; IF v_cnt >
0 THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Room is already reserved for these dates'; END IF; END
IF; END
```

trg_res_validate_bu BEFORE UPDATE on hotel_reservations

The same five rules on update, so a confirmed booking cannot be changed into an invalid one.

```
BEGIN DECLARE v_room_status VARCHAR(20); DECLARE v_cap INT; DECLARE v_ts DATE; DECLARE v_te DATE; DECLARE v_cnt INT; IF NEW.status <> 'cancelled' THEN -- room must exist, not be under maintenance, and fit the guests SELECT r.status, rt.max_occupancy INTO v_room_status, v_cap FROM rooms r JOIN room_types rt ON rt.room_type_id = r.room_type_id AND rt.hotel_id = r.hotel_id WHERE r.room_id = NEW.room_id; IF v_room_status = 'maintenance' THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Room is under maintenance and cannot be reserved'; END IF; IF NEW.num_guests > v_cap THEN
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Number of guests exceeds the room type capacity'; END IF; -- stay must lie inside the trip dates (check-out may be the day after the last day) SELECT start_date, end_date INTO v_ts, v_te FROM trips WHERE trip_id = NEW.trip_id; IF NEW.check_in < v_ts OR NEW.check_out > DATE_ADD(v_te, INTERVAL 1 DAY) THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Stay must be inside the trip dates'; END IF; -- only an owner or editor of the trip may book SELECT COUNT(*) INTO v_cnt FROM trip_members WHERE trip_id = NEW.trip_id AND user_id = NEW.booked_by AND role IN ('owner','editor'); IF v_cnt = 0 THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Only a trip owner or editor can book a room'; END IF; -- no double booking of the same room on overlapping nights SELECT COUNT(*) INTO v_cnt FROM hotel_reservations h WHERE h.room_id = NEW.room_id AND h.status <> 'cancelled' AND h.reservation_id <> COALESCE(NEW.reservation_id, 0) AND NEW.check_in < h.check_out AND NEW.check_out > h.check_in; IF v_cnt > 0 THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Room is already reserved for these dates'; END IF; END IF; END
```

trg_hreview_completed_bi BEFORE INSERT on hotel_reviews

A stay can only be reviewed once its reservation is `completed`. This stops reviews for stays that were never taken.

```
BEGIN DECLARE v_status VARCHAR(20); SELECT status INTO v_status FROM hotel_reservations WHERE reservation_id = NEW.reservation_id; IF v_status IS NULL OR v_status <> 'completed' THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Only a completed stay can be reviewed'; END IF; END
```

trg_hreview_completed_bu BEFORE UPDATE on hotel_reviews

Same rule on update, so a review cannot be re-pointed at an unfinished stay.

```
BEGIN DECLARE v_status VARCHAR(20); SELECT status INTO v_status FROM hotel_reservations WHERE reservation_id = NEW.reservation_id; IF v_status IS NULL OR v_status <> 'completed' THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Only a completed stay can be reviewed'; END IF; END
```

trg_hotels_place_type_bi BEFORE INSERT on hotels

A `hotels` row is a subtype of `places`, so the place it points at must have `place_type = 'Hotel'`. A foreign key cannot express that.

```
BEGIN DECLARE v_type VARCHAR(40); SELECT place_type INTO v_type FROM places WHERE place_id = NEW.hotel_id; IF v_type IS NULL OR v_type <> 'Hotel' THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'A hotel row must point at a place whose place_type is Hotel'; END IF; END
```

trg_hotels_place_type_bu BEFORE UPDATE on hotels

Same rule on update, keeping the subtype link honest.

```
BEGIN DECLARE v_type VARCHAR(40); SELECT place_type INTO v_type FROM places WHERE place_id = NEW.hotel_id; IF v_type IS NULL OR v_type <> 'Hotel' THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'A hotel row must point at a place whose place_type is Hotel'; END IF; END
```

B. Transport triggers — 03_transport_module.sql, 12 triggers

trg_transport_validate_bi BEFORE INSERT on transport_bookings

Six rules for a transport booking: carrier type must suit the mode (an airline cannot run a cab); flight, train and bus need both stations; origin and destination must differ; station type must suit the mode; a linked itinerary item must be from the same trip; and the booker must be the trip owner or an editor.

```
BEGIN DECLARE v_ctype VARCHAR(20); DECLARE v_exp_ctype VARCHAR(20); DECLARE v_otype VARCHAR(20); DECLARE
v_dtype VARCHAR(20); DECLARE v_exp_st VARCHAR(20); DECLARE v_item_trip INT; DECLARE v_cnt INT; -- 1. the
carrier type must suit the mode (an airline cannot run a cab) SELECT carrier_type INTO v_ctype FROM
carriers WHERE carrier_id = NEW.carrier_id; SET v_exp_ctype = CASE NEW.transport_mode WHEN 'flight' THEN
'airline' WHEN 'train' THEN 'railway' WHEN 'cab' THEN 'cab_operator' WHEN 'bus' THEN 'bus_operator' ELSE
'metro_operator' END; IF v_ctype <> v_exp_ctype THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Carrier
type does not match the transport mode'; END IF; -- 2. every mode except cab runs between two different
stations of the matching kind IF NEW.transport_mode <> 'cab' THEN IF NEW.origin_station_id IS NULL OR
NEW.destination_station_id IS NULL THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'This mode needs an
origin and a destination station'; END IF; IF NEW.origin_station_id = NEW.destination_station_id THEN
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Origin and destination must be different'; END IF; SET
v_exp_st = CASE NEW.transport_mode WHEN 'flight' THEN 'airport' WHEN 'train' THEN 'railway_station' WHEN
'bus' THEN 'bus_stand' ELSE 'metro_station' END; SELECT station_type INTO v_otype FROM stations WHERE
station_id = NEW.origin_station_id; SELECT station_type INTO v_dtype FROM stations WHERE station_id =
NEW.destination_station_id; IF v_otype <> v_exp_st OR v_dtype <> v_exp_st THEN
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Station type does not match the transport mode'; END IF; END
IF; -- 3. a linked itinerary item must belong to the same trip IF NEW.item_id IS NOT NULL THEN SELECT
d.trip_id INTO v_item_trip FROM itinerary_items i JOIN itinerary_days d ON d.day_id = i.day_id WHERE
i.item_id = NEW.item_id; IF v_item_trip <> NEW.trip_id THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT =
'Itinerary item belongs to a different trip'; END IF; END IF; -- 4. only an owner or editor of the trip
may book SELECT COUNT(*) INTO v_cnt FROM trip_members WHERE trip_id = NEW.trip_id AND user_id =
NEW.booked_by AND role IN ('owner','editor'); IF v_cnt = 0 THEN
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Only a trip owner or editor can book transport'; END IF; END
```

trg_transport_validate_bu BEFORE UPDATE on transport_bookings

The same six rules on update.

```
BEGIN DECLARE v_ctype VARCHAR(20); DECLARE v_exp_ctype VARCHAR(20); DECLARE v_otype VARCHAR(20); DECLARE
v_dtype VARCHAR(20); DECLARE v_exp_st VARCHAR(20); DECLARE v_item_trip INT; DECLARE v_cnt INT; -- 1. the
carrier type must suit the mode (an airline cannot run a cab) SELECT carrier_type INTO v_ctype FROM
carriers WHERE carrier_id = NEW.carrier_id; SET v_exp_ctype = CASE NEW.transport_mode WHEN 'flight' THEN
'airline' WHEN 'train' THEN 'railway' WHEN 'cab' THEN 'cab_operator' WHEN 'bus' THEN 'bus_operator' ELSE
'metro_operator' END; IF v_ctype <> v_exp_ctype THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Carrier
type does not match the transport mode'; END IF; -- 2. every mode except cab runs between two different
stations of the matching kind IF NEW.transport_mode <> 'cab' THEN IF NEW.origin_station_id IS NULL OR
NEW.destination_station_id IS NULL THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'This mode needs an
origin and a destination station'; END IF; IF NEW.origin_station_id = NEW.destination_station_id THEN
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Origin and destination must be different'; END IF; SET
v_exp_st = CASE NEW.transport_mode WHEN 'flight' THEN 'airport' WHEN 'train' THEN 'railway_station' WHEN
'bus' THEN 'bus_stand' ELSE 'metro_station' END; SELECT station_type INTO v_otype FROM stations WHERE
station_id = NEW.origin_station_id; SELECT station_type INTO v_dtype FROM stations WHERE station_id =
NEW.destination_station_id; IF v_otype <> v_exp_st OR v_dtype <> v_exp_st THEN
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Station type does not match the transport mode'; END IF; END
IF; -- 3. a linked itinerary item must belong to the same trip IF NEW.item_id IS NOT NULL THEN SELECT
d.trip_id INTO v_item_trip FROM itinerary_items i JOIN itinerary_days d ON d.day_id = i.day_id WHERE
i.item_id = NEW.item_id; IF v_item_trip <> NEW.trip_id THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT =
'Itinerary item belongs to a different trip'; END IF; END IF; -- 4. only an owner or editor of the trip
may book SELECT COUNT(*) INTO v_cnt FROM trip_members WHERE trip_id = NEW.trip_id AND user_id =
NEW.booked_by AND role IN ('owner','editor'); IF v_cnt = 0 THEN
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Only a trip owner or editor can book transport'; END IF; END
```

trg_flight_mode_bi BEFORE INSERT on flight_details

A `flight_details` row must point at a booking whose mode is `flight`. One trigger per mode keeps each subtype tied to its parent.

```
BEGIN DECLARE v_mode VARCHAR(10); SELECT transport_mode INTO v_mode FROM transport_bookings WHERE
transport_id = NEW.transport_id; IF v_mode IS NULL OR v_mode <> 'flight' THEN SIGNAL SQLSTATE '45000' SET
MESSAGE_TEXT = 'flight_details requires a booking whose mode is flight'; END IF; END
```

trg_flight_mode_bu BEFORE UPDATE on flight_details

Same rule on update.

```
BEGIN DECLARE v_mode VARCHAR(10); SELECT transport_mode INTO v_mode FROM transport_bookings WHERE transport_id = NEW.transport_id; IF v_mode IS NULL OR v_mode <> 'flight' THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'flight_details requires a booking whose mode is flight'; END IF; END
```

trg_train_mode_bi BEFORE INSERT on train_details

A `train_details` row must point at a booking whose mode is `train`.

```
BEGIN DECLARE v_mode VARCHAR(10); SELECT transport_mode INTO v_mode FROM transport_bookings WHERE transport_id = NEW.transport_id; IF v_mode IS NULL OR v_mode <> 'train' THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'train_details requires a booking whose mode is train'; END IF; END
```

trg_train_mode_bu BEFORE UPDATE on train_details

Same rule on update.

```
BEGIN DECLARE v_mode VARCHAR(10); SELECT transport_mode INTO v_mode FROM transport_bookings WHERE transport_id = NEW.transport_id; IF v_mode IS NULL OR v_mode <> 'train' THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'train_details requires a booking whose mode is train'; END IF; END
```

trg_cab_mode_bi BEFORE INSERT on cab_details

A `cab_details` row must point at a booking whose mode is `cab`.

```
BEGIN DECLARE v_mode VARCHAR(10); SELECT transport_mode INTO v_mode FROM transport_bookings WHERE transport_id = NEW.transport_id; IF v_mode IS NULL OR v_mode <> 'cab' THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'cab_details requires a booking whose mode is cab'; END IF; END
```

trg_cab_mode_bu BEFORE UPDATE on cab_details

Same rule on update.

```
BEGIN DECLARE v_mode VARCHAR(10); SELECT transport_mode INTO v_mode FROM transport_bookings WHERE transport_id = NEW.transport_id; IF v_mode IS NULL OR v_mode <> 'cab' THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'cab_details requires a booking whose mode is cab'; END IF; END
```

trg_bus_mode_bi BEFORE INSERT on bus_details

A `bus_details` row must point at a booking whose mode is `bus`.

```
BEGIN DECLARE v_mode VARCHAR(10); SELECT transport_mode INTO v_mode FROM transport_bookings WHERE transport_id = NEW.transport_id; IF v_mode IS NULL OR v_mode <> 'bus' THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'bus_details requires a booking whose mode is bus'; END IF; END
```

trg_bus_mode_bu BEFORE UPDATE on bus_details

Same rule on update.

```
BEGIN DECLARE v_mode VARCHAR(10); SELECT transport_mode INTO v_mode FROM transport_bookings WHERE transport_id = NEW.transport_id; IF v_mode IS NULL OR v_mode <> 'bus' THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'bus_details requires a booking whose mode is bus'; END IF; END
```

trg_metro_mode_bi BEFORE INSERT on metro_details

A `metro_details` row must point at a booking whose mode is `metro`.

```
BEGIN DECLARE v_mode VARCHAR(10); SELECT transport_mode INTO v_mode FROM transport_bookings WHERE transport_id = NEW.transport_id; IF v_mode IS NULL OR v_mode <> 'metro' THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'metro_details requires a booking whose mode is metro'; END IF; END
```

trg_metro_mode_bu BEFORE UPDATE on metro_details

Same rule on update.

```
BEGIN DECLARE v_mode VARCHAR(10); SELECT transport_mode INTO v_mode FROM transport_bookings WHERE transport_id = NEW.transport_id; IF v_mode IS NULL OR v_mode <> 'metro' THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'metro_details requires a booking whose mode is metro'; END IF; END
```

C. Audit and alert triggers — 05_smart_features.sql, 7 triggers

trg_trips_audit_au AFTER UPDATE on trips

Writes an audit row when a trip's `budget_total` or `status` changes. Unchanged columns are skipped, so the log stays meaningful.

```
BEGIN IF NOT (OLD.budget_total <=> NEW.budget_total) THEN INSERT INTO audit_log (table_name, record_id, action, field_name, old_value, new_value, changed_by) VALUES ('trips', NEW.trip_id, 'UPDATE', 'budget_total', CAST(OLD.budget_total AS CHAR), CAST(NEW.budget_total AS CHAR), USER()); END IF; IF NOT (OLD.status <=> NEW.status) THEN INSERT INTO audit_log (table_name, record_id, action, field_name, old_value, new_value, changed_by) VALUES ('trips', NEW.trip_id, 'UPDATE', 'status', CAST(OLD.status AS CHAR), CAST(NEW.status AS CHAR), USER()); END IF; END
```

trg_res_audit_au AFTER UPDATE on hotel_reservations

Writes an audit row when a reservation's `status` changes.

```
BEGIN IF NOT (OLD.status <=> NEW.status) THEN INSERT INTO audit_log (table_name, record_id, action, field_name, old_value, new_value, changed_by) VALUES ('hotel_reservations', NEW.reservation_id, 'UPDATE', 'status', CAST(OLD.status AS CHAR), CAST(NEW.status AS CHAR), USER()); END IF; END
```

trg_transport_audit_au AFTER UPDATE on transport_bookings

Writes an audit row when a transport booking's `status` changes.

```
BEGIN IF NOT (OLD.status <=> NEW.status) THEN INSERT INTO audit_log (table_name, record_id, action, field_name, old_value, new_value, changed_by) VALUES ('transport_bookings', NEW.transport_id, 'UPDATE', 'status', CAST(OLD.status AS CHAR), CAST(NEW.status AS CHAR), USER()); END IF; END
```

trg_exp_audit_au AFTER UPDATE on expenses

Writes an audit row when an expense `amount` changes.

```
BEGIN IF NOT (OLD.amount <=> NEW.amount) THEN INSERT INTO audit_log (table_name, record_id, action, field_name, old_value, new_value, changed_by) VALUES ('expenses', NEW.expense_id, 'UPDATE', 'amount', CAST(OLD.amount AS CHAR), CAST(NEW.amount AS CHAR), USER()); END IF; END
```

trg_exp_audit_ad AFTER DELETE on expenses

Writes an audit row when an expense is deleted, so spending history is not lost silently.

```
BEGIN INSERT INTO audit_log (table_name, record_id, action, field_name, old_value, new_value, changed_by) VALUES ('expenses', OLD.expense_id, 'DELETE', 'amount', CAST(OLD.amount AS CHAR), NULL, USER()); END
```

trg_exp_budget_alert_ai AFTER INSERT on expenses

After an expense is added, raises a `budget_80` alert at 80 percent of the budget and a `budget_exceeded` alert once it is passed. `INSERT IGNORE` plus a unique key means each alert type is recorded once per trip, not once per expense.

```
BEGIN DECLARE v_budget DECIMAL(10,2); DECLARE v_spent DECIMAL(12,2); SELECT budget_total INTO v_budget FROM trips WHERE trip_id = NEW.trip_id; SELECT COALESCE(SUM(amount), 0) INTO v_spent FROM expenses WHERE trip_id = NEW.trip_id; IF v_budget > 0 THEN IF v_spent > v_budget THEN INSERT IGNORE INTO trip_alerts (trip_id, alert_type, message) VALUES (NEW.trip_id, 'budget_exceeded', CONCAT('Spending ', v_spent, ' has exceeded the budget of ', v_budget)); ELSEIF v_spent >= 0.8 * v_budget THEN INSERT IGNORE INTO trip_alerts (trip_id, alert_type, message) VALUES (NEW.trip_id, 'budget_80', CONCAT('Spending ', v_spent, ' has reached 80 percent of the budget of ', v_budget)); END IF; END IF; END
```

trg_exp_budget_alert_au AFTER UPDATE on expenses

The same check after an expense is edited, so correcting an amount also corrects the alerts.

```
BEGIN DECLARE v_budget DECIMAL(10,2); DECLARE v_spent DECIMAL(12,2); SELECT budget_total INTO v_budget
FROM trips WHERE trip_id = NEW.trip_id; SELECT COALESCE(SUM(amount), 0) INTO v_spent FROM expenses WHERE
trip_id = NEW.trip_id; IF v_budget > 0 THEN IF v_spent > v_budget THEN INSERT IGNORE INTO trip_alerts
(trip_id, alert_type, message) VALUES (NEW.trip_id, 'budget_exceeded', CONCAT('Spending ', v_spent, ' has
exceeded the budget of ', v_budget)); ELSEIF v_spent >= 0.8 * v_budget THEN INSERT IGNORE INTO trip_alerts
(trip_id, alert_type, message) VALUES (NEW.trip_id, 'budget_80', CONCAT('Spending ', v_spent, ' has
reached 80 percent of the budget of ', v_budget)); END IF; END IF; END
```

3. CHECK constraints (27)

These run before a row is written. A violation raises error 3819. They are grouped by table so a reviewer can see everything protecting one table in one place.

Constraint	Rule	What it protects
`bookings` `chk_bookings_amount`	`(\`amount\` >= 0)`	A booking amount cannot be negative.
`cab_details` `chk_cab_distance`	`((\`distance_km\` is null) or (\`distance_km\` > 0))`	Distance, when given, must be greater than zero.
`destination_season_months` `chk_dsm_month`	`(\`month_no\` between 1 and 12)`	A season month must be 1 to 12.
`expenses` `chk_expenses_amount`	`(\`amount\` > 0)`	An expense must be greater than zero, so no silent empty rows.
`flight_details` `chk_flight_baggage`	`(\`baggage_kg\` >= 0)`	Baggage weight cannot be negative.
`hotel_payments` `chk_pay_amount`	`(\`amount\` > 0)`	A payment must be greater than zero.
`hotel_payments` `chk_pay_no`	`(\`payment_no\` >= 1)`	Payment numbering starts at 1.
`hotel_reservations` `chk_res_dates`	`(\`check_out\` > \`check_in\`)`	Check-out must be after check-in.
`hotel_reservations` `chk_res_guests`	`(\`num_guests\` >= 1)`	At least one guest.
`hotel_reservations` `chk_res_rate`	`(\`rate_per_night\` >= 0)`	A nightly rate cannot be negative.
`hotel_reviews` `chk_hrev_rating`	`(\`rating\` between 1 and 5)`	A hotel rating is 1 to 5.
`hotels` `chk_hotels_star`	`(\`star_rating\` between 1 and 5)`	Star rating is 1 to 5.
`itinerary_days` `chk_days_num`	`(\`day_number\` >= 1)`	Day numbering starts at 1.
`itinerary_items` `chk_items_cost`	`(\`estimated_cost\` >= 0)`	An estimated cost cannot be negative.
`itinerary_items` `chk_items_seq`	`(\`seq_no\` >= 1)`	Sequence numbers start at 1.
`itinerary_items` `chk_items_time`	`((\`start_time\` is null) or (\`end_time\` is null) or (\`end_time\` > \`start_time\`))`	An item cannot end before it starts. Untimed items pass.
`metro_details` `chk_metro_stops`	`((\`num_stops\` is null) or (\`num_stops\` >= 1))`	Stop count, when given, must be at least one.
`reviews` `chk_reviews_rating`	`(\`rating\` between 1 and 5)`	A destination rating is 1 to 5.
`room_types` `chk_rt_occ`	`(\`max_occupancy\` >= 1)`	A room type holds at least one guest.
`room_types` `chk_rt_rate`	`(\`base_rate\` >= 0)`	A base rate cannot be negative.

`rooms` `chk_rooms_floor`	`(`floor_no` >= 0)`	Floor number cannot be negative.
`transport_bookings` `chk_tb_fare`	`(`fare` >= 0)`	A fare cannot be negative.
`transport_bookings` `chk_tb_times`	`(`arrive_at` > `depart_at`)`	Arrival must be after departure.
`transport_passengers` `chk_tp_age`	`((`age` is null) or (`age` <= 120))`	An age above 120 is rejected as a data-entry error.
`transport_passengers` `chk_tp_no`	`(`passenger_no` >= 1)`	Passenger numbering starts at 1.
`trips` `chk_trips_budget`	`(`budget_total` >= 0)`	A budget cannot be negative.
`trips` `chk_trips_dates`	`(`end_date` >= `start_date`)`	A trip cannot end before it starts.

4. ENUM columns (22)

An ENUM restricts a column to a fixed list. This is how the schema keeps status values and categories consistent without a lookup table, and it makes an invalid status impossible to store in the first place.

Column	Allowed values
`audit_log.action`	UPDATE · DELETE
`bookings.status`	pending · confirmed · cancelled
`bus_details.bus_type`	city_bus · ordinary · ac_seater · sleeper · volvo_ac
`cab_details.trip_type`	one_way · round_trip · local
`cab_details.vehicle_type`	hatchback · sedan · suv · tempo_traveller
`carriers.carrier_type`	airline · railway · cab_operator · bus_operator · metro_operator
`flight_details.cabin_class`	economy · premium_economy · business · first
`hotel_payments.method`	card · upi · netbanking · cash
`hotel_payments.status`	success · failed · refunded
`hotel_reservations.status`	pending · confirmed · cancelled · checked_in · completed
`itinerary_items.status`	planned · done · skipped
`metro_details.ticket_type`	single · return · day_pass · smart_card
`room_types.bed_type`	single · twin · double · queen · king
`rooms.status`	available · maintenance
`stations.station_type`	airport · railway_station · bus_stand · metro_station
`train_details.travel_class`	2S · SL · CC · 3A · 2A · 1A
`transport_bookings.status`	pending · confirmed · cancelled · completed
`transport_bookings.transport_mode`	flight · train · cab · bus · metro
`trip_alerts.alert_type`	budget_80 · budget_exceeded
`trip_members.role`	owner · editor · viewer
`trips.status`	planned · ongoing · completed · cancelled

`users.role`	user · admin

5. UNIQUE constraints (19)

These stop duplicates. Several are business rules rather than plain keys, for example one booking per itinerary item, one room number per hotel, and one alert of each type per trip.

Constraint	Table	What it stops
`uk_actcat_name`	`activity_categories`	Activity category names are unique.
`uk_amenities_name`	`amenities`	Amenity names are unique.
`uk_bookings_item`	`bookings`	One booking per itinerary item.
`uk_carriers_name`	`carriers`	Carrier names are unique.
`uk_dest_name_country`	`destinations`	A destination name is unique within its country.
`uk_expcat_name`	`expense_categories`	Expense category names are unique.
`uk_res_code`	`hotel_reservations`	Confirmation codes are unique.
`uk_days_date`	`itinerary_days`	Two days of a trip cannot share a date.
`uk_days_num`	`itinerary_days`	Day numbers are unique within a trip.
`uk_items_day_seq`	`itinerary_items`	Sequence numbers are unique within a day.
`uk_places_dest_name`	`places`	A place name is unique within its destination.
`uk_rt_hotel_name`	`room_types`	Room type names are unique within a hotel.
`uk_rt_id_hotel`	`room_types`	A room type belongs to exactly one hotel.
`uk_rooms_hotel_number`	`rooms`	A room number is unique within its hotel.
`uk_stations_type_code`	`stations`	A station code is unique per station type.
`uk_tb_code`	`transport_bookings`	Transport booking references are unique.
`uk_tp_seat`	`transport_passengers`	A seat is held by one passenger on a booking.
`uk_alert_trip_type`	`trip_alerts`	One alert of each type per trip, so repeated expenses do not spam the table.
`uk_users_email`	`users`	Two accounts cannot share an email address.

6. Foreign keys (42)

Each foreign key states what happens to the child row when the parent is deleted. The split is deliberate: **CASCADE 27**, **RESTRICT 11**, **SET NULL 4**. The three meanings, which the table below refers to by name:

Rule	Meaning
CASCADE	Delete the child too. Used for parts of an aggregate: delete a trip and its days, items, bookings and alerts go with it.
RESTRICT	Refuse the delete. Used for reference data and owners: a destination that places depend on, or a user who owns trips, cannot vanish.
SET NULL	Keep the child, clear the link. Used for optional context: an expense survives its itinerary item being deleted.

Rule	Child column	Parent	Constraint
CASCADE	`bookings.item_id`	`itinerary_items(item_id)`	`fk_bookings_ite`
CASCADE	`bus_details.transport_id`	`transport_bookings(transport_id)`	`fk_bus_tb`
CASCADE	`cab_details.transport_id`	`transport_bookings(transport_id)`	`fk_cab_tb`
CASCADE	`destination_season_months.destination_id`	`destinations(destination_id)`	`fk_dsm_dest`
RESTRICT	`expenses.expense_category_id`	`expense_categories(expense_category_id)`	`fk_exp_category`
SET NULL	`expenses.item_id`	`itinerary_items(item_id)`	`fk_exp_item`
RESTRICT	`expenses.paid_by`	`users(user_id)`	`fk_exp_payer`
CASCADE	`expenses.trip_id`	`trips(trip_id)`	`fk_exp_trip`
CASCADE	`flight_details.transport_id`	`transport_bookings(transport_id)`	`fk_flight_tb`
CASCADE	`hotel_amenities.amenity_id`	`amenities(amenity_id)`	`fk_ha_amenity`
CASCADE	`hotel_amenities.hotel_id`	`hotels(hotel_id)`	`fk_ha_hotel`
CASCADE	`hotel_payments.reservation_id`	`hotel_reservations(reservation_id)`	`fk_pay_res`
RESTRICT	`hotel_reservations.booked_by`	`users(user_id)`	`fk_res_booker`
RESTRICT	`hotel_reservations.room_id`	`rooms(room_id)`	`fk_res_room`
CASCADE	`hotel_reservations.trip_id`	`trips(trip_id)`	`fk_res_trip`
CASCADE	`hotel_reviews.reservation_id`	`hotel_reservations(reservation_id)`	`fk_hrev_res`
CASCADE	`hotels.hotel_id`	`places(place_id)`	`fk_hotels_place`
CASCADE	`itinerary_days.trip_id`	`trips(trip_id)`	`fk_days_trip`
RESTRICT	`itinerary_items.category_id`	`activity_categories(category_id)`	`fk_items_catego`
CASCADE	`itinerary_items.day_id`	`itinerary_days(day_id)`	`fk_items_day`
SET NULL	`itinerary_items.place_id`	`places(place_id)`	`fk_items_place`
CASCADE	`metro_details.transport_id`	`transport_bookings(transport_id)`	`fk_metro_tb`
RESTRICT	`places.destination_id`	`destinations(destination_id)`	`fk_places_desti`
CASCADE	`reviews.destination_id`	`destinations(destination_id)`	`fk_rev_dest`
CASCADE	`reviews.user_id`	`users(user_id)`	`fk_rev_user`

CASCADE	`room_types.hotel_id`	`hotels(hotel_id)`	`fk_rt_hotel`
CASCADE	`rooms.room_type_id, rooms.hotel_id`	`room_types(room_type_id, hotel_id)`	`fk_rooms_type_h`
CASCADE	`saved_destinations.destination_id`	`destinations(destination_id)`	`fk_sd_dest`
CASCADE	`saved_destinations.user_id`	`users(user_id)`	`fk_sd_user`
CASCADE	`train_details.transport_id`	`transport_bookings(transport_id)`	`fk_train_tb`
RESTRICT	`transport_bookings.booked_by`	`users(user_id)`	`fk_tb_booker`
RESTRICT	`transport_bookings.carrier_id`	`carriers(carrier_id)`	`fk_tb_carrier`
RESTRICT	`transport_bookings.destination_station_id`	`stations(station_id)`	`fk_tb_dest`
SET NULL	`transport_bookings.item_id`	`itinerary_items(item_id)`	`fk_tb_item`
RESTRICT	`transport_bookings.origin_station_id`	`stations(station_id)`	`fk_tb_origin`
CASCADE	`transport_bookings.trip_id`	`trips(trip_id)`	`fk_tb_trip`
CASCADE	`transport_passengers.transport_id`	`transport_bookings(transport_id)`	`fk_tp_tb`
SET NULL	`transport_passengers.user_id`	`users(user_id)`	`fk_tp_user`
CASCADE	`trip_alerts.trip_id`	`trips(trip_id)`	`fk_alert_trip`
CASCADE	`trip_members.trip_id`	`trips(trip_id)`	`fk_tm_trip`
CASCADE	`trip_members.user_id`	`users(user_id)`	`fk_tm_user`
RESTRICT	`trips.owner_id`	`users(user_id)`	`fk_trips_owner`

7. NOT NULL coverage

Beyond the named constraints, **149 columns** are declared `NOT NULL`. This is the quietest layer of protection and often the most effective: a missing value is rejected before any CHECK or trigger is consulted.

8. How to see the rules fire

Each demo script already contains a test for its own layer. Run the database first, then:

Script	What it demonstrates
`02_demo_queries.sql`	14 rule tests (C1–C14) covering the data-integrity triggers and CHECK constraints from script 01.
`04_transport_demo.sql`	10 rule tests (X1–X10) covering the transport triggers, including the carrier/mode mismatch and the station rules.
`06_demo_amenities_views_triggers.sql`	The audit triggers and both budget alerts, inside a transaction that is rolled back.

> **Note on a rejected write.** Every validation trigger uses `SIGNAL SQLSTATE '45000'`, which surfaces as MySQL error **1644** with the trigger's message text. A CHECK violation surfaces as error **3819**. Seeing those two codes in the test output is the proof that the rule fired.